

Open QHPC Software Ecosystem (openQSE)

openQSE Community
shehataa@ornl.gov

ORNL is managed by UT-Battelle LLC for the US Department of Energy

Administrative

- **QCUF 2026:** July 20th – 23rd
- **openQSE:** July 24th, 2026 – 4 working groups official launch
- **openQSE confluence** instance is up. If you don't have an account ping me
- **On going efforts** (Goal: Create milestones that drive the initiative forward)
 - Architecture SC 2026 paper. Nearing completion
 - Architecture expanded journal
 - System Integration Paper. Led by Johannes Blaschke. Currently active
 - Compiler survey paper. Led by Josh Moles & Guen Prawiroatmodjo
 - <https://forms.gle/mCA8QxFgmsBQ5jcC6>
 - QRMI/QDMI alignment effort
- **Upcoming speakers:** Quantinuum (April 13th), IISc (April 27th)
- **Looking for speakers** to cover various topics: control electronics research, energy efficiency, compiler deep dives, system integration.
- Soon to transition weekly meetings towards: speakers to aid in covering the software stack requirements, working group reports and feedback
- **Upcoming Events:** QCE 26, SC 26 (IBM and ORNL workshops accepted)

openQSE Phased Approach

Build a **community** and **working groups**

- Drive progress through collaborative, focused efforts
- github.com/openqse
- openqse.org
- openqse.atlassian.net/wiki

Identify and **organize** software layers

- Clarify boundaries and responsibilities across the stack
- Investigate existing solutions

Define software **component** categories

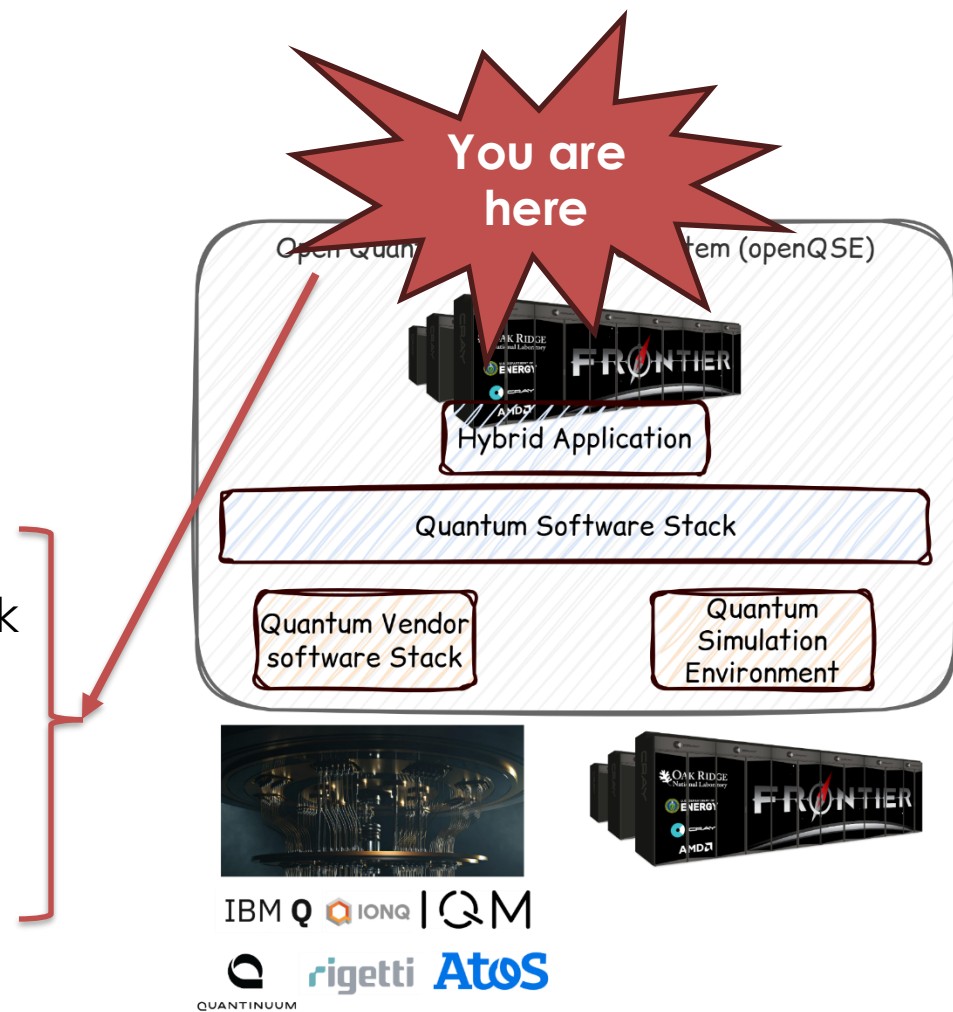
- Establish a common language to facilitate design discussions

Determine which layers require **exposed interfaces**

- Focus design efforts where interoperability is critical

Specify the **interfaces** and their semantics

- Enable targeted innovation and modular development

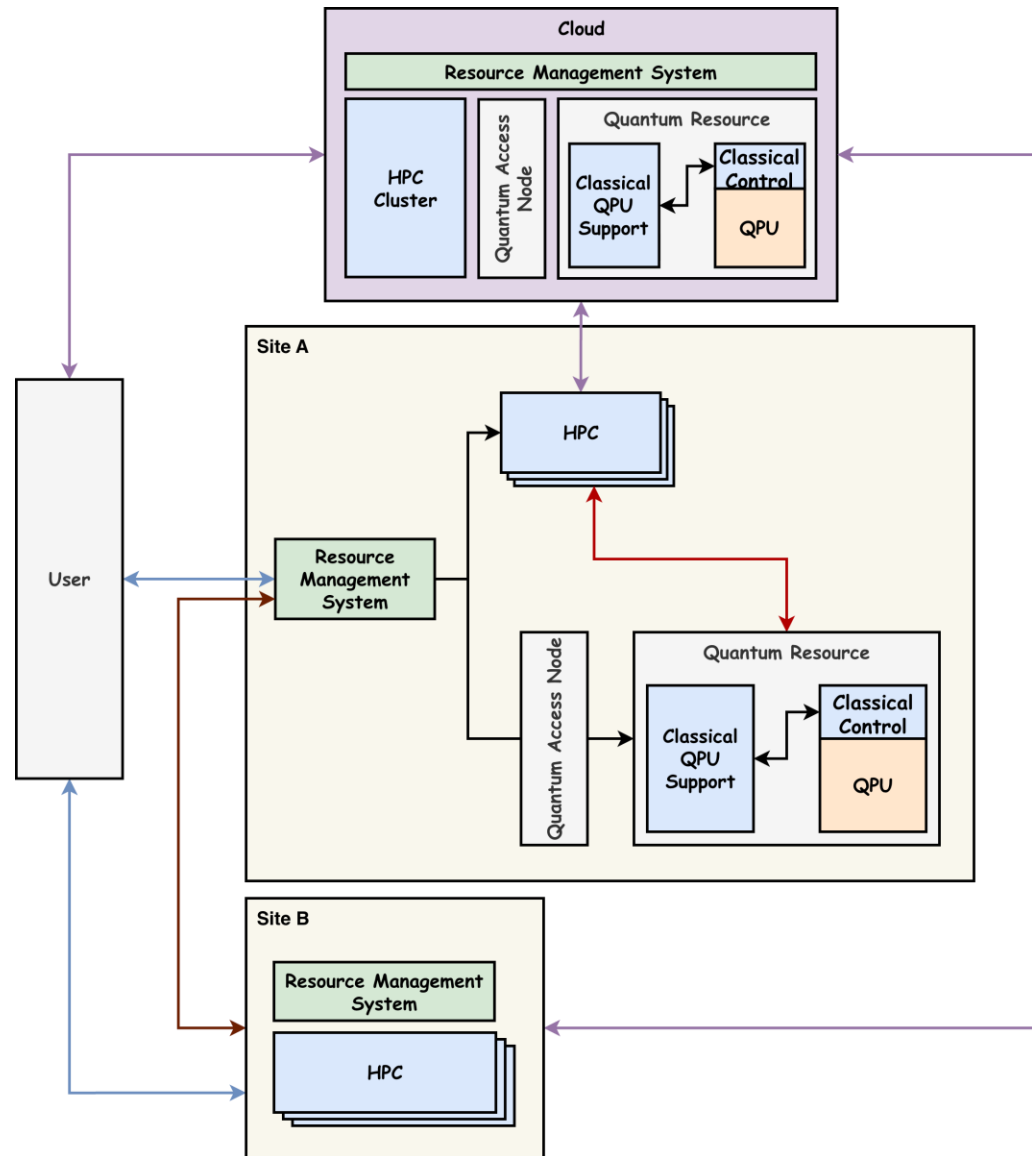


Many stacks to power them all.
Many teams to advance them.
One ecosystem to connect them all
and in open source align them.

Four Classification Axes

- **Deployment Model:** Cloud, On-prem, Federated
- **Quantum Hardware Modality:** Superconducting, Ion-traps, neutral atoms ...
- **Application Interaction Patterns:** Workflow- and Accelerator-centric
- **Targeted Quantum Computing Era:** NISQ, FTQC

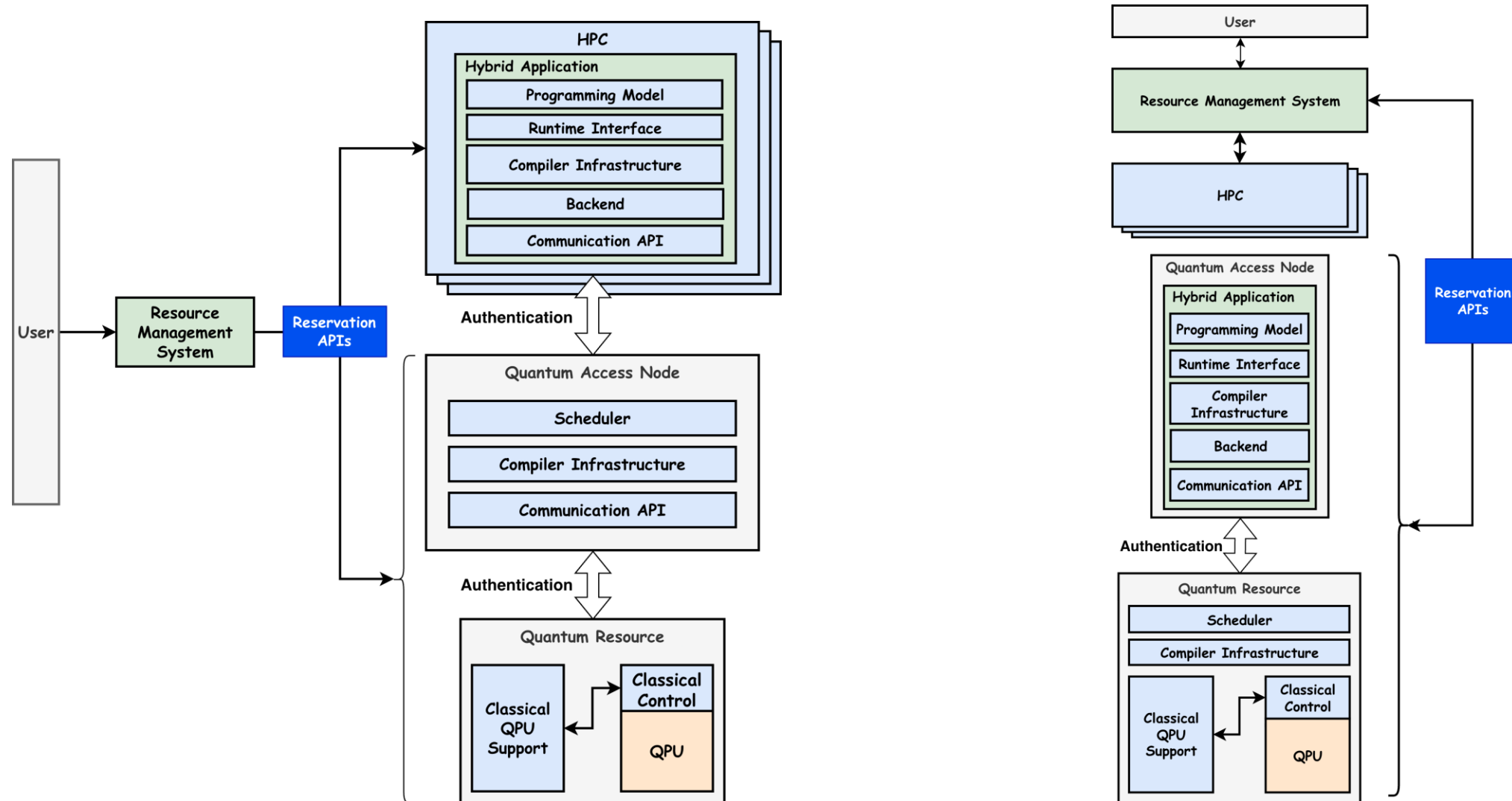
Deployment Model



Quantum Hardware Modality

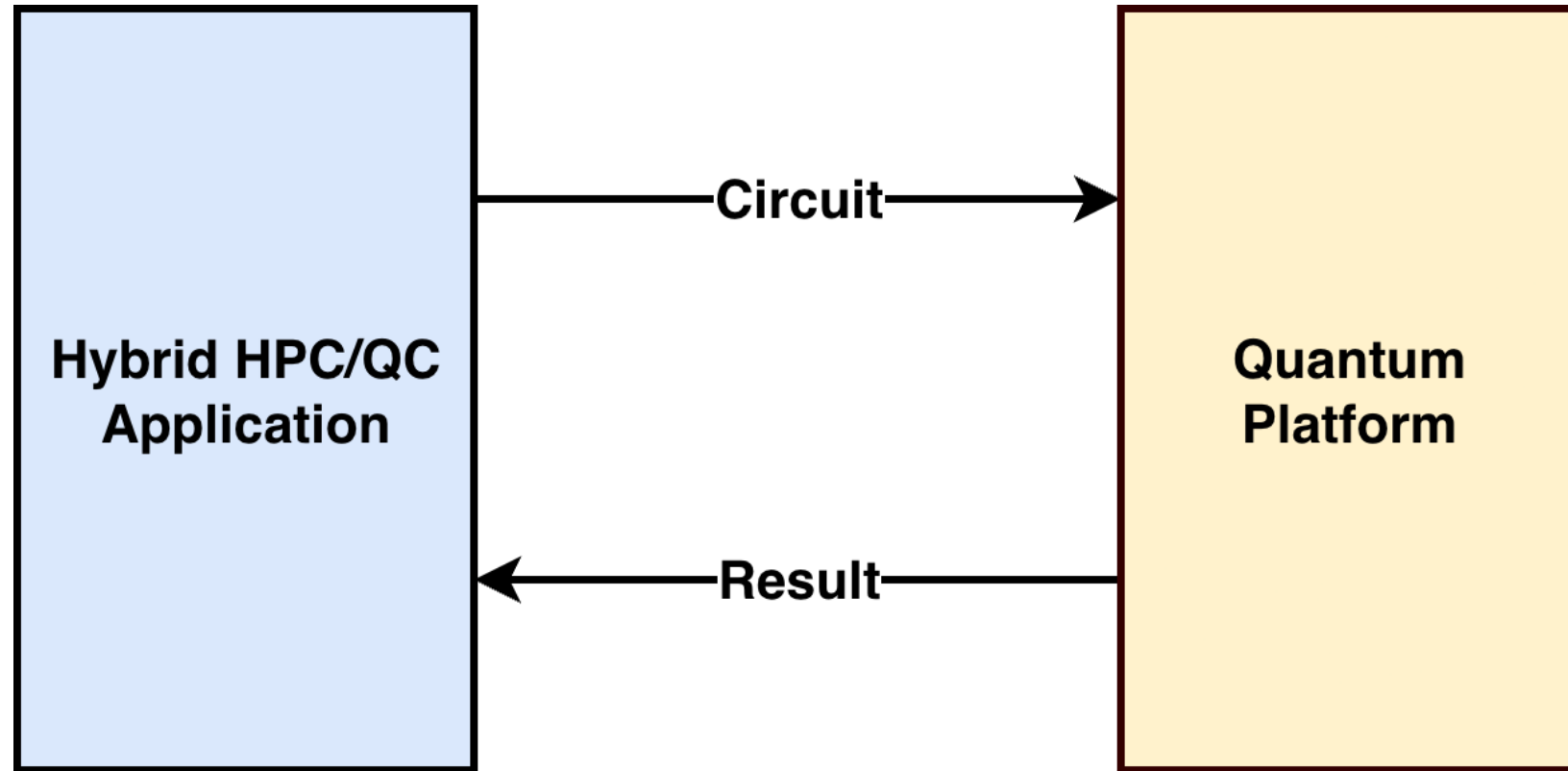
- Multiple quantum hardware modalities (superconducting, ions, photonic, etc.)
- Each modality imposes distinct **latency**, **control**, and **orchestration** constraints
- **QEC** introduces strict timing requirements tied to **qubit coherence**
- Most current **stacks** target a **subset of modalities** for optimization simplicity
- **Cross-modality** support remains limited in today's software ecosystems
- **Scalability** requires abstraction layers decoupling software from hardware
- **Compiler** and **toolchains** must adapt to modality-specific constraints
- **Consistent upper-layer interfaces** needed despite hardware diversity

Design Space: Application Interaction Patterns



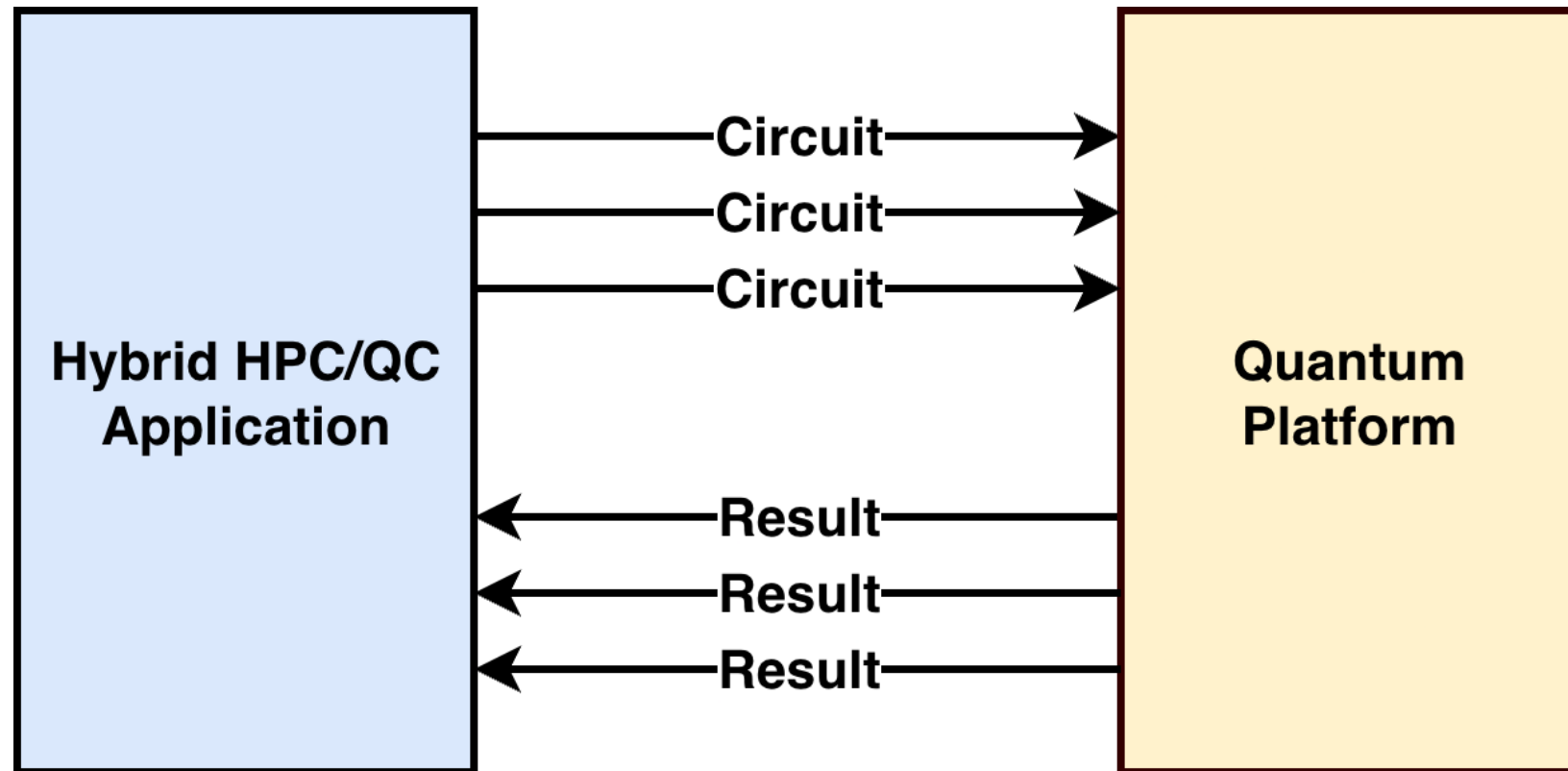
Design Space: Application Behavior

One Deep Circuit



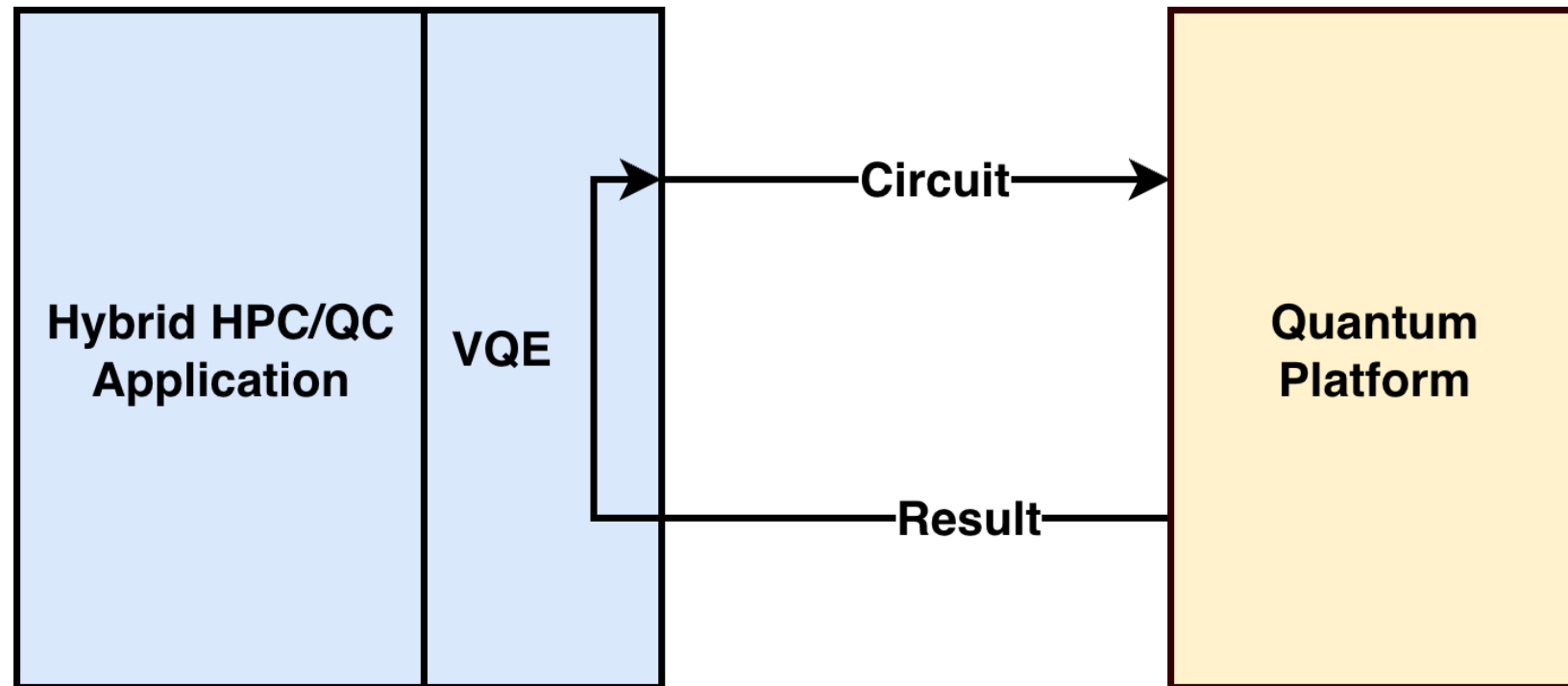
Design Space: Application Behavior

Ensemble of Shallow/Deep Independent Circuits



Design Space: Application Behavior

Iterative optimization loop

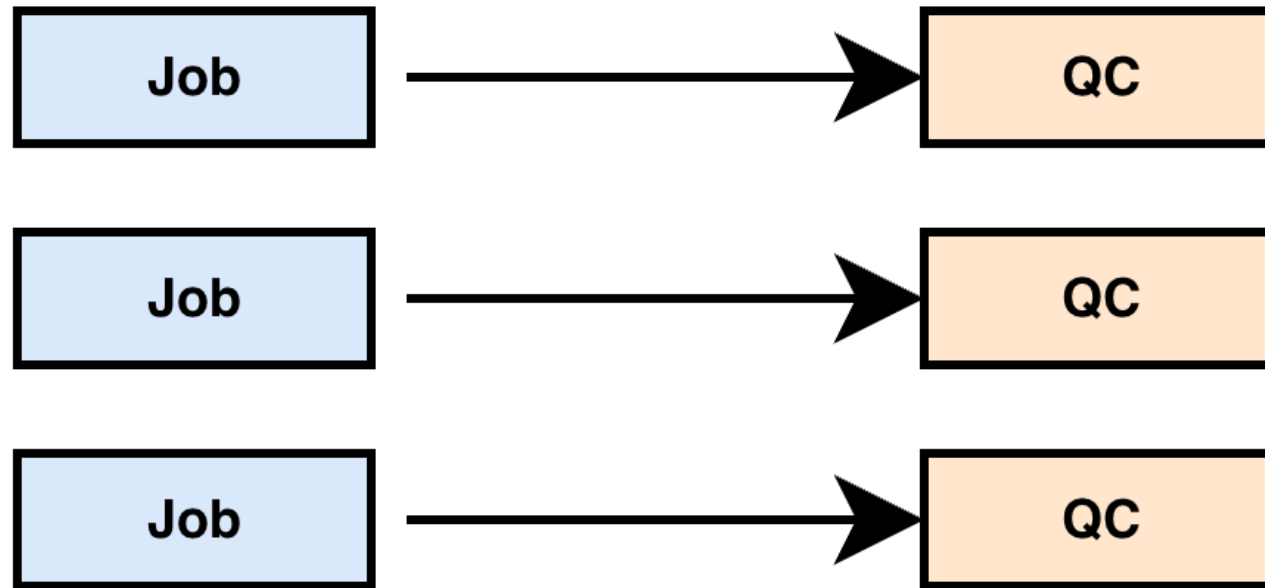


Design Space: Quantum Resource Usage

Dedicated Quantum Resource per Hybrid Job

Application

QC Sim/HW

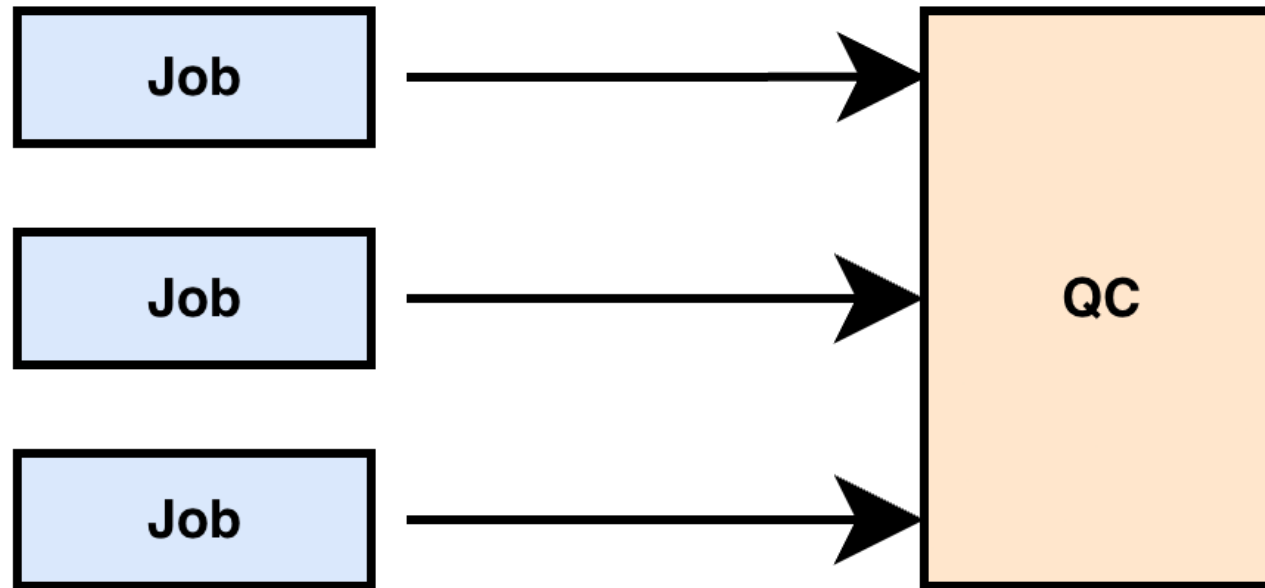


Design Space: Quantum Resource Usage

One Quantum Resource for all Hybrid Job

Application

QC Sim/HW

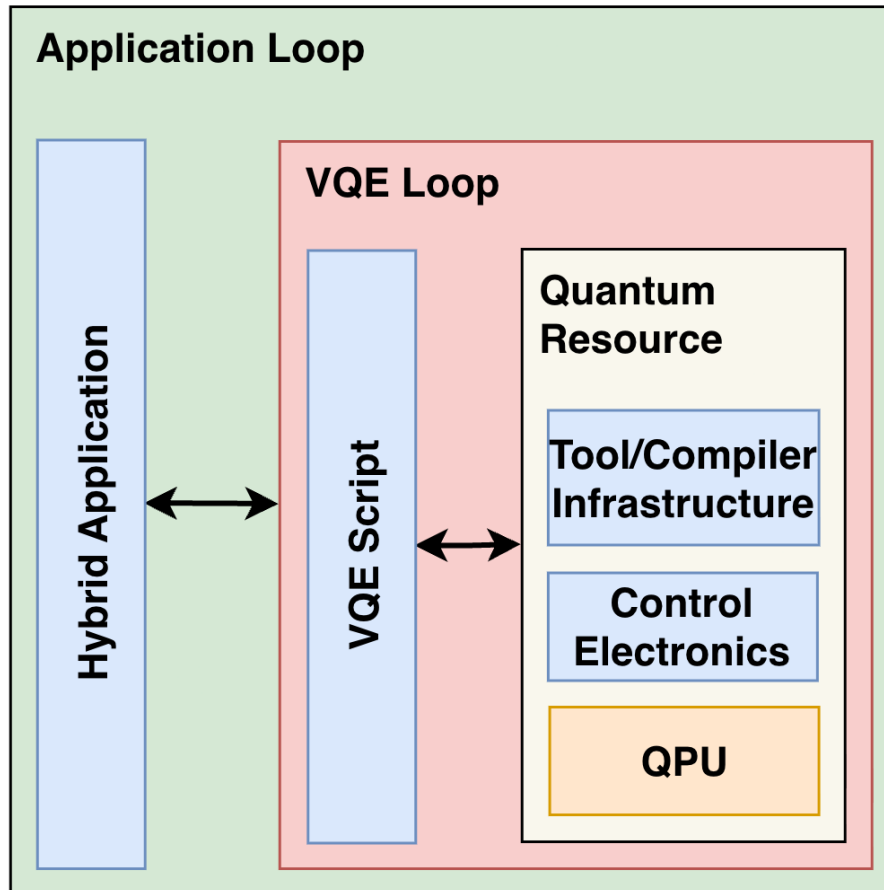


Quantum Computing Era

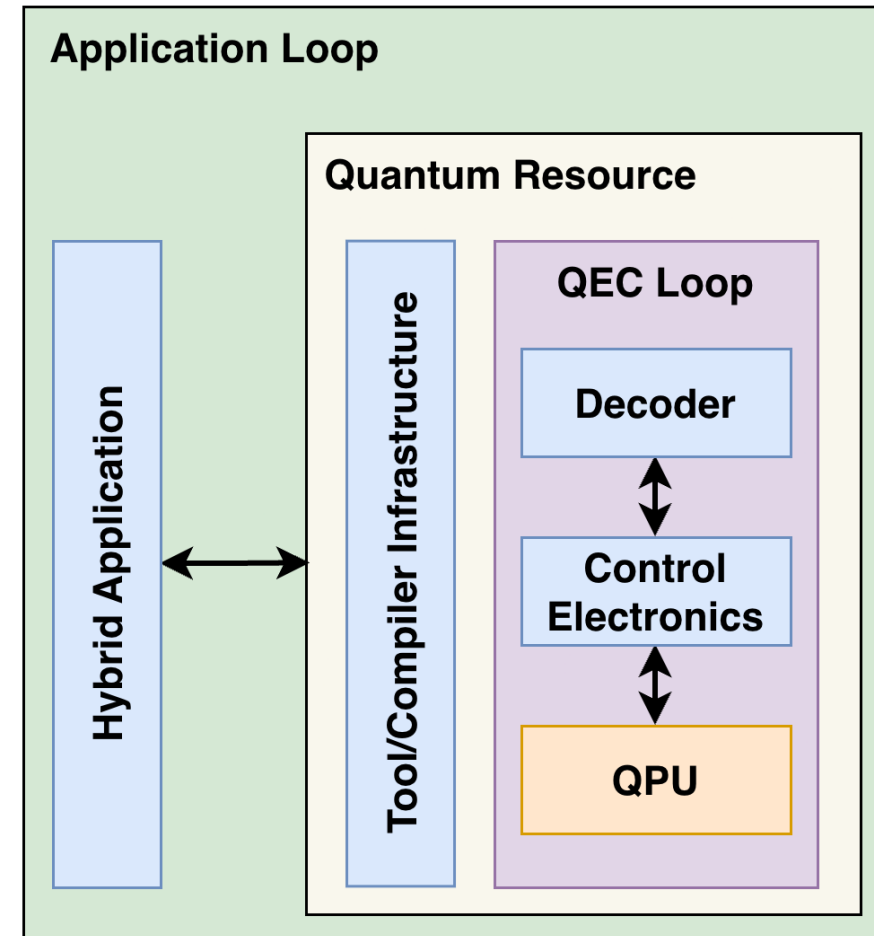
- NISQ and FTQC systems will coexist for the foreseeable future. Or will they?
- NISQ: hybrid workloads (e.g., VQE), latency-sensitive but not timing-critical
- FTQC: inner QEC loops with strict timing constraints and correctness impact
- Abstract application layer from hardware-specific timing details
- Expose hardware capabilities via stable, queryable APIs
- Maintain architectural stability while enabling FTQC evolution

Design Space: Future Evolution

Noisy Intermediate Scale Quantum Computing (NISQ)

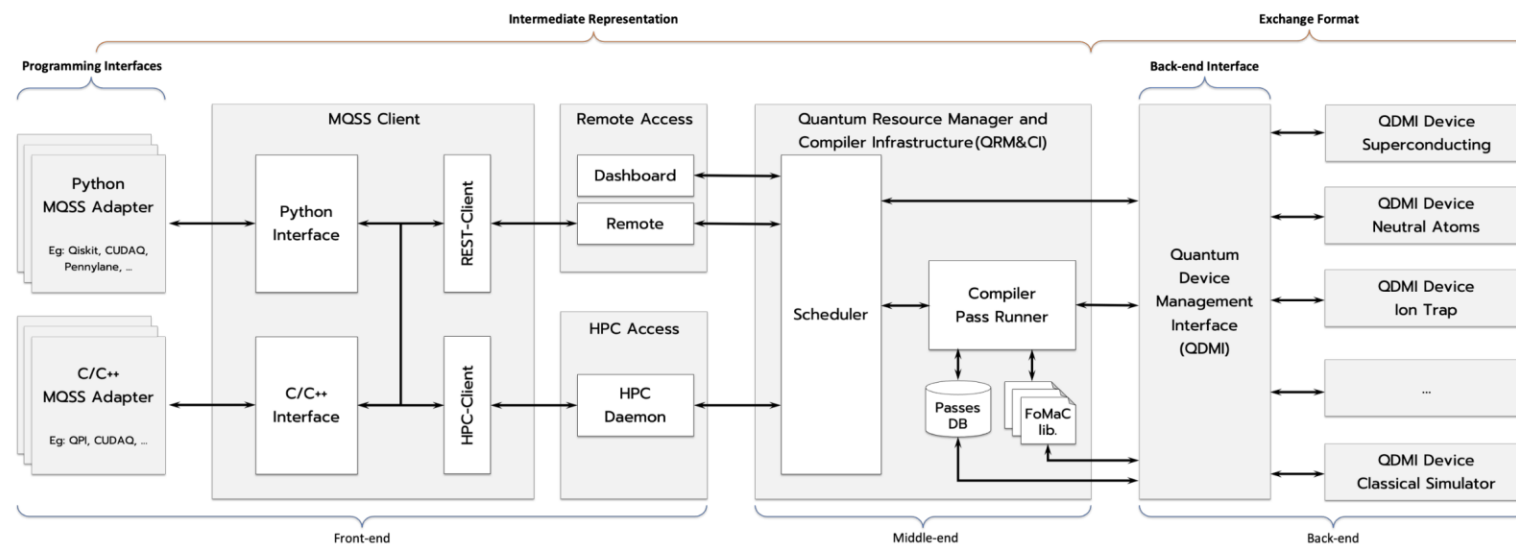
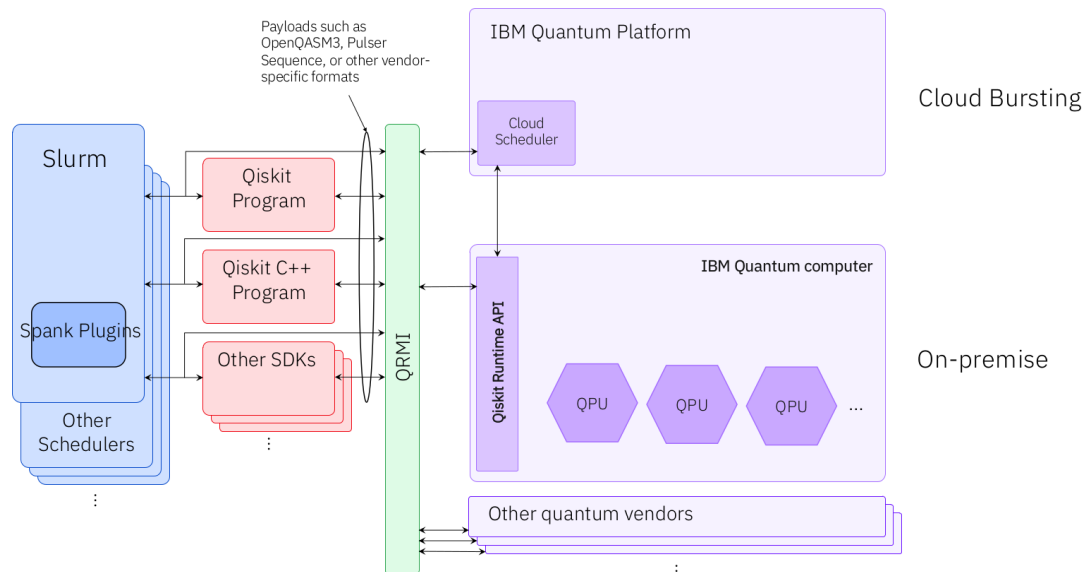


Fault-Tolerant Quantum Computing (FTQC)

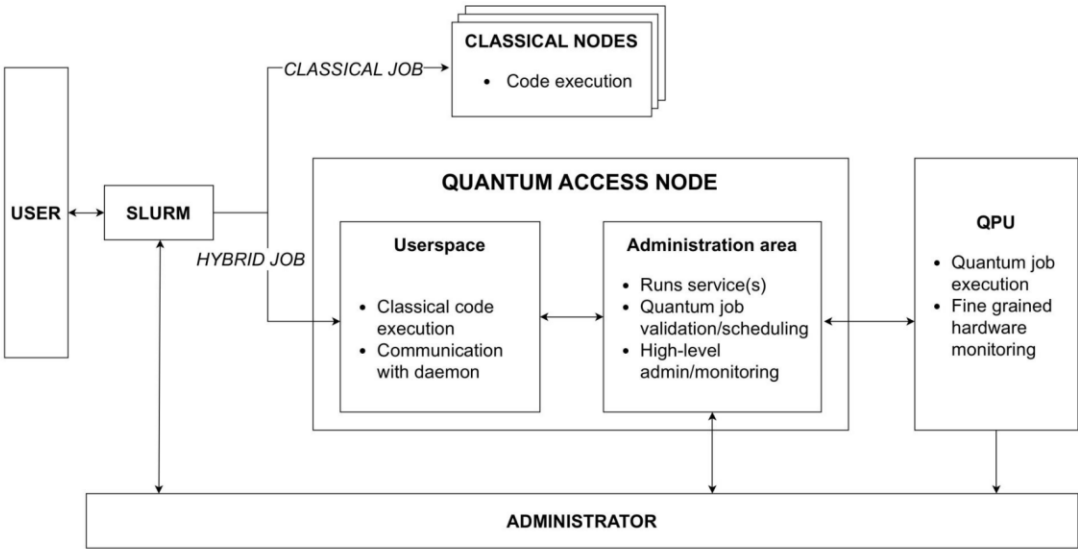
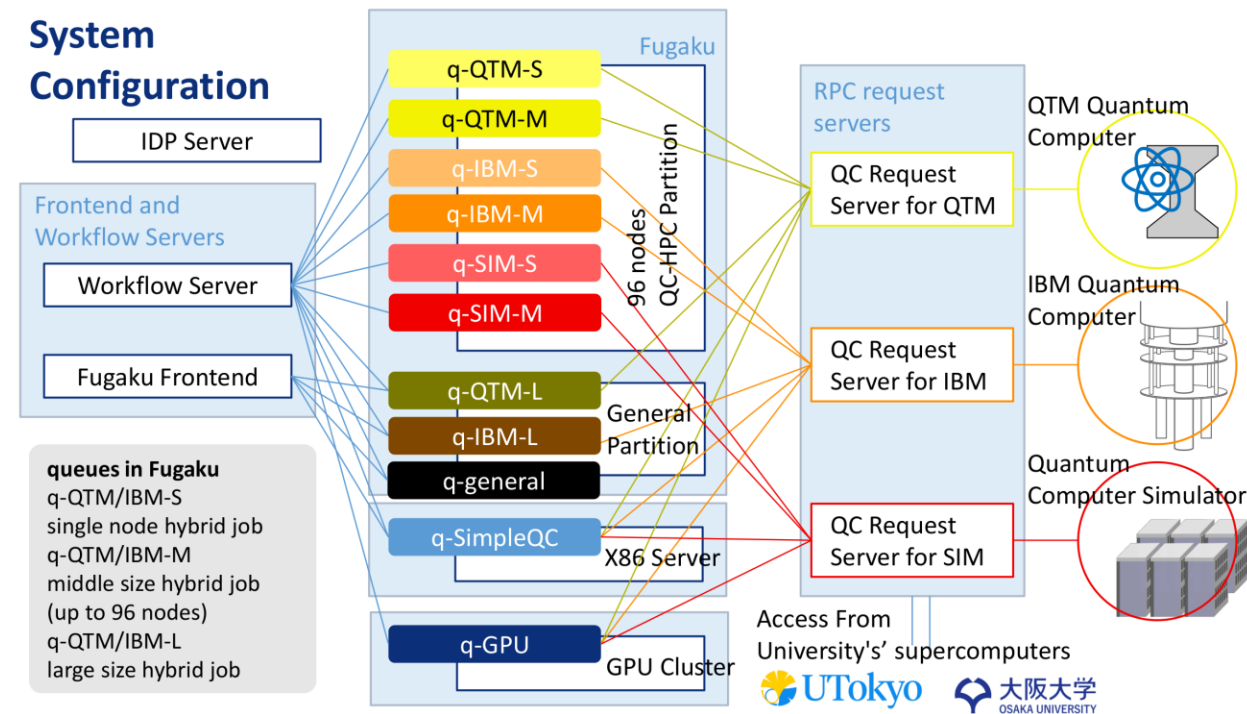


Representative Stacks

QRMI / Slurm



Representative Stacks



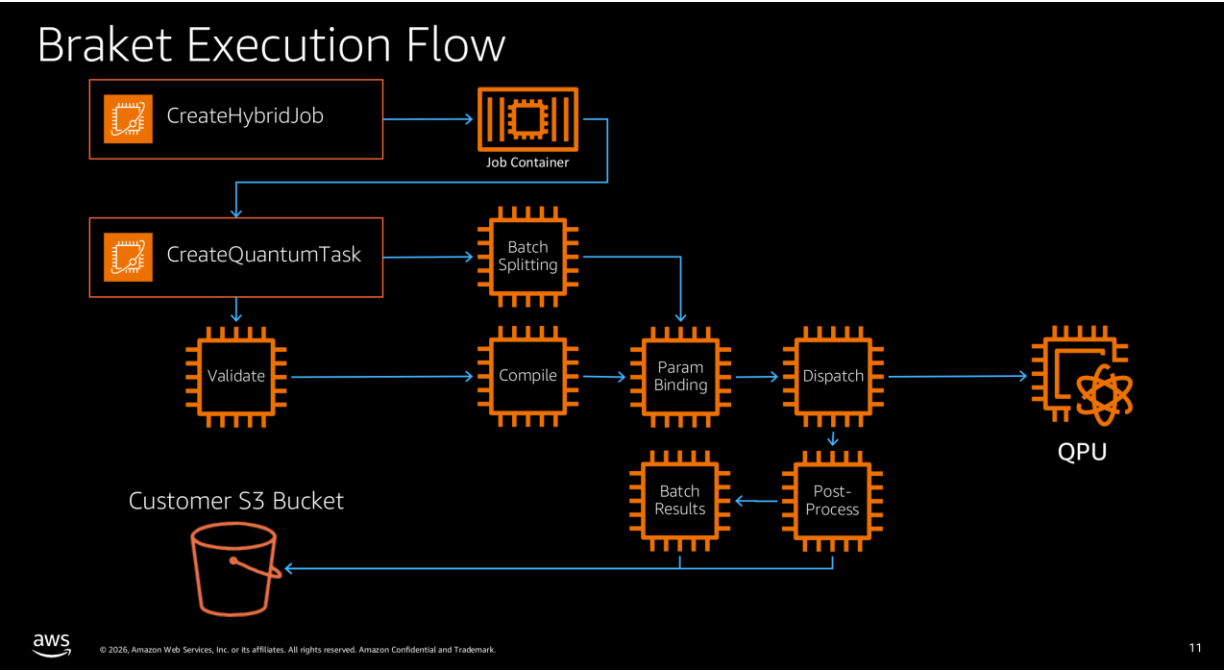
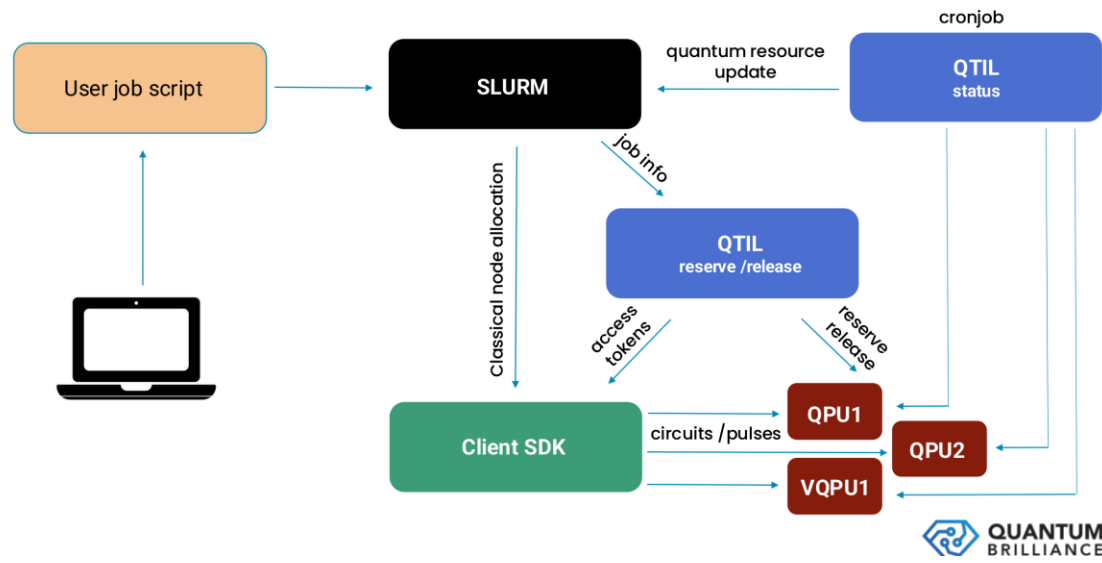
pasqal SAS 2026

A. Wennersteen et al., "Towards a user-centric HPC-QC environment", 2025, In Proc. Of the SC Workshops '25), ACM ArXiv: 2509.20525

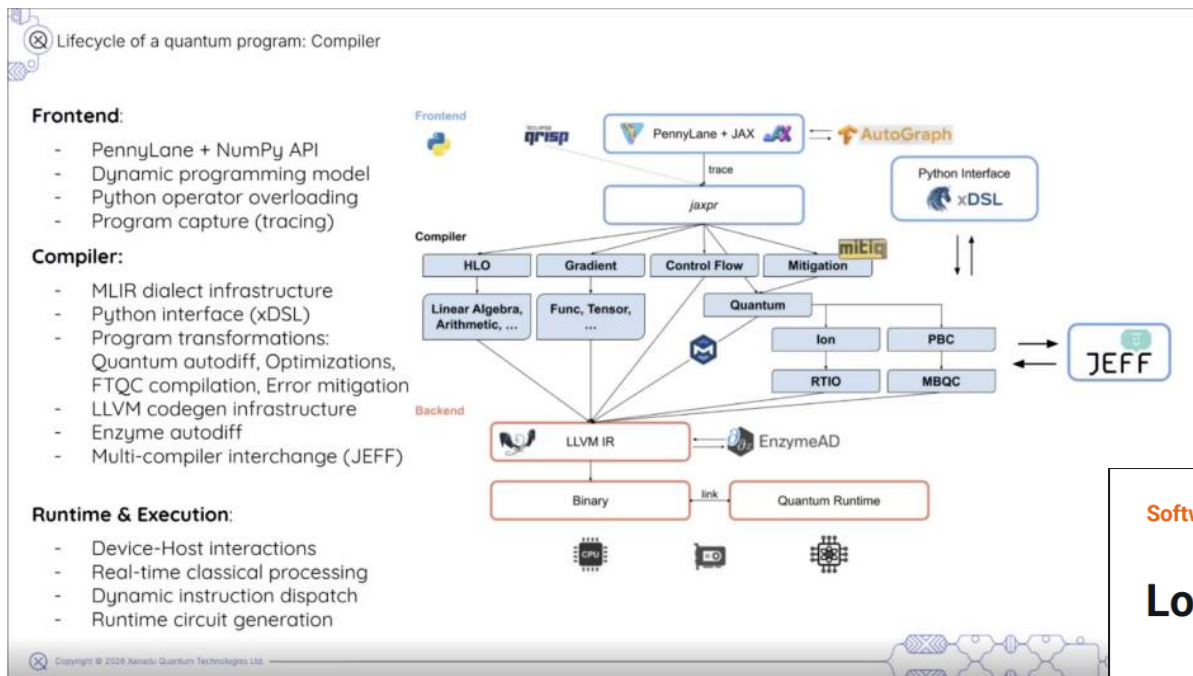
Representative Stacks

- My my

QTL + SLURM in a nutshell

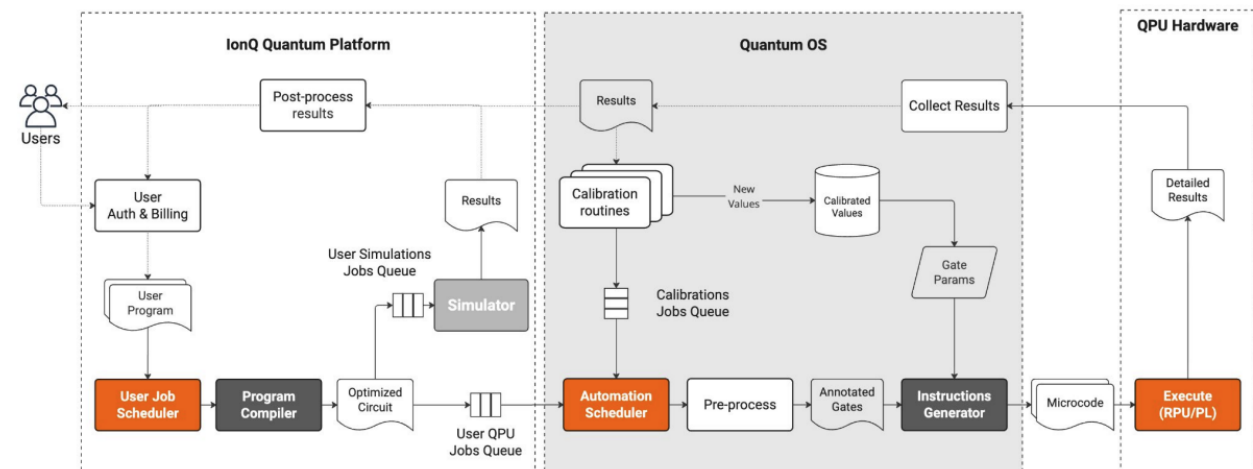


Representative Stacks

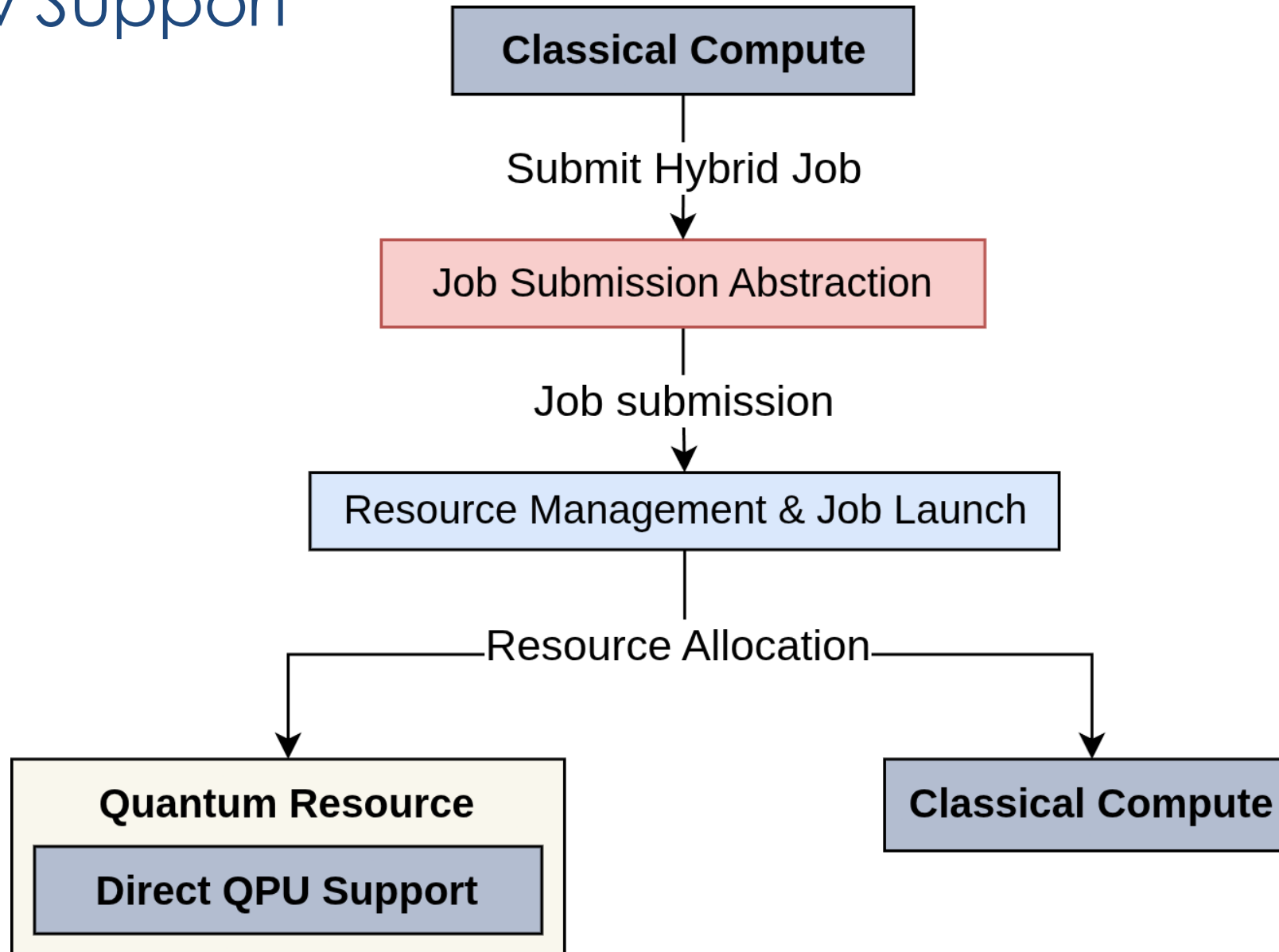


Software Stack and Throughput

Loosely-coupled Services and Functions



Workflow Support



Design Patterns

- **Layered architecture** separating applications, runtime, compilation, and hardware
- **Stable upper interfaces** with vendor specialization near the device
- **Explicit runtime boundaries** for submission, status, and results
- **Multi-stage program lowering** with backend-specific optimization points
- **Integrated scheduling**, reservations, and capability discovery
- **Built-in telemetry** for execution state, system health, and debugging
- **Support for hybrid** workflows across HPC and quantum resources

Design Patterns

- **Common runtime API** to decouple applications from stack-specific SDKs
- **Composable compilation** and transformation pipelines across frameworks
- **Abstracted connection** and **transport management** across environments
- **Bidirectional communication** between runtime and quantum-side services
- **Unified, policy-aware** multi-tenant scheduling models
- **Standardized access** to system state and calibration data
- **Structured exposure** of hardware capabilities and control features
- **Support for hardware-near** classical processing and feedback loops

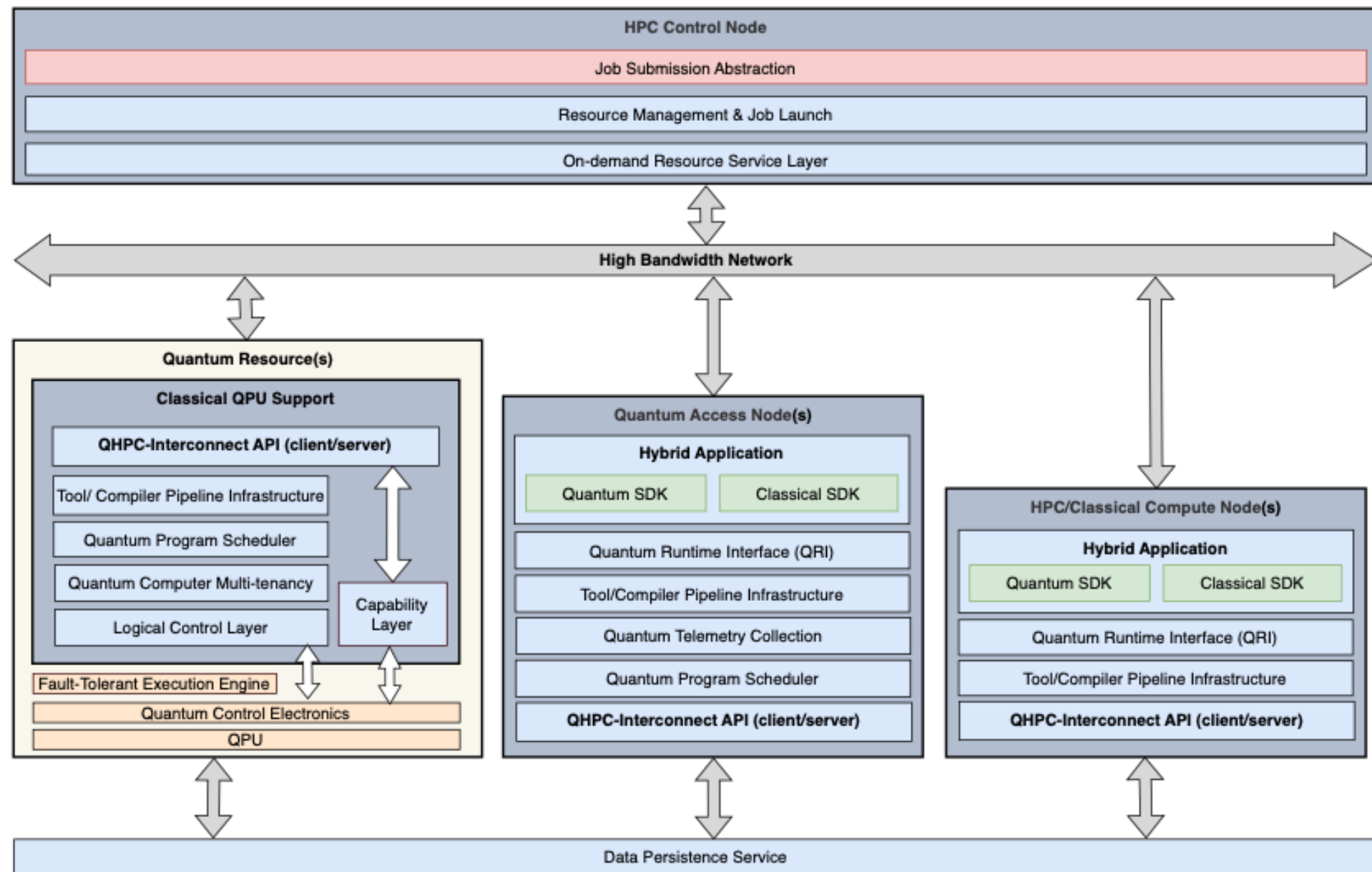
Stack Coverage

Arch.	Runtime Interfaces	Resource Interface	Scheduling	Compiler	QHPC-Interconnect	FTQC & Control
AWS	AWS SDK	QDMI , Managed (Braket)	Managed jobs; Slurm	SDK + service-native	Backend-dependent	Pulse access emerging
IBM	QRMI ; Qiskit SDK; IBM Quantum Platform (cloud)	QRMI resource API	Slurm (SPANK, GRES)	Qiskit compiler stack	Backend-dependent	FTQC planned [5]
IonQ	QDMI , QRMI ; cloud API	QDMI , QRMI	Cloud-based Fair-share; calibration-aware	Multi-pass; native gates	Backend-dependent	Microcode/RPU; timing exposed. FTQC planned
JHPC-Q	SQC Broker, Multi-SDK support	Broker abstraction	HPC-priority scheduler	Qiskit, TKET, CUDA-Q	gRPC & Backend-dependent	Planned FTQC mgmt
MQSS	Multi-SDK support	QDMI	Slurm + MQT Predictor	MLIR-based toolchain	Backend-dependent	Planned FTQC mgmt
Pasqal	QRMI ; Pulser, Qiskit, CUDA-Q	QRMI	QRMI + Slurm	Pulser, Qiskit	Backend-dependent	FTQC planned
QB	QTIL API; SDK	QTIL API	Slurm + QTIL alloc	QCStack transpilation	Backend-dependent	Waveform/calibration
Quantinuum	Nexus, CUDA-Q, QIR	Proprietary	Fair-share queue	Guppy, TKET	gRPC & Tierkreis	FTQC active; proprietary
Xanadu	PennyLane; Catalyst	Host-managed	External scheduler	Catalyst (MLIR/JAX)	Backend-dependent	QEC in Catalyst [65]

TABLE I

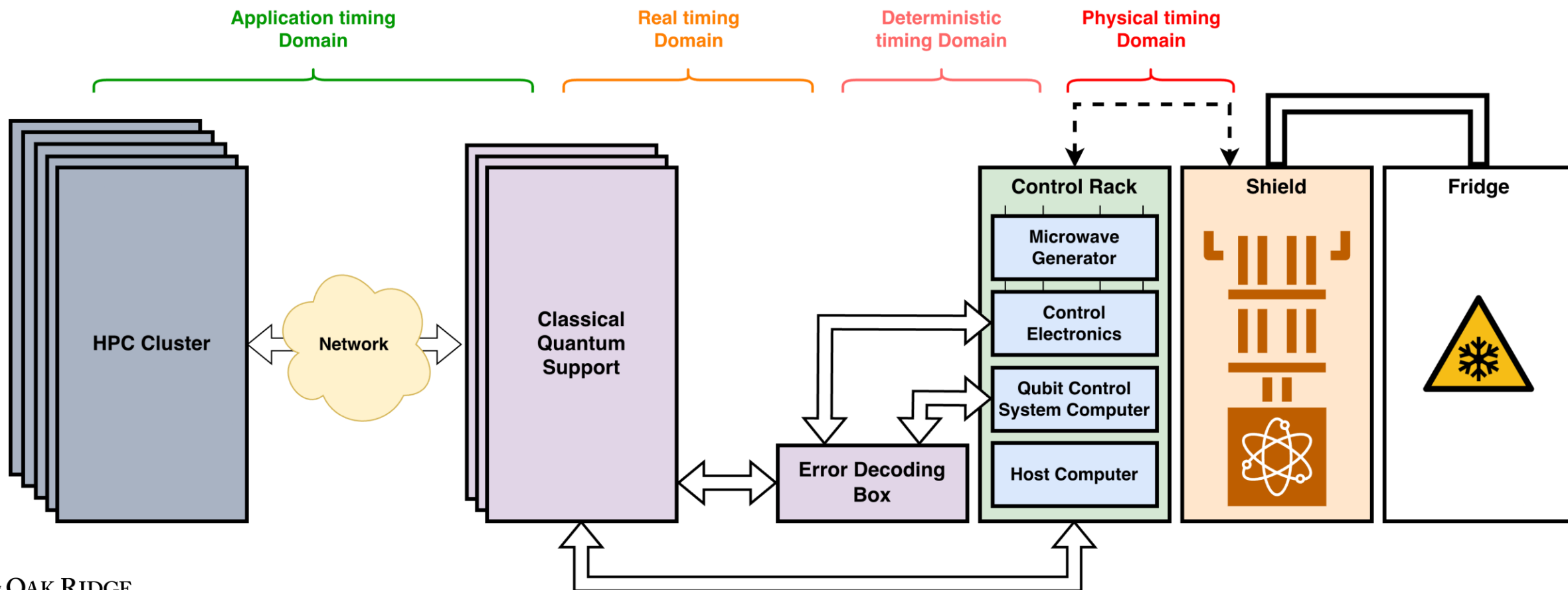
MAPPING OF SURVEYED ARCHITECTURES AS OF APRIL 2026 TO THE LAYERS IN FIGURE 2. **BOLD** DENOTES RECURRING INTERFACES (QDMI, QRMI). **YELLOW** MARKS EMERGING SUPPORT.

openQSE: Architecture



Quantum at ORNL

- **Purple** is future
- Much to explore: tighter integration, resource management, system timing, etc



Next Steps

- Refine Architecture
 - Longer journal article
 - Detail each stack
 - Detail requirements
 - Detail Terms
 - Detail Architecture
 - Prototype
 - Repeat
 - Implement and validate on ORNL deployment as part of QSC
- Thoughts? Comments?